异步派生记忆的逻辑删除质量：写入口、完成验证与协议演化的 SQLite 工程案例

Logical Deletion Quality for Asynchronously Derived Memory: A SQLite Engineering Case Study of Write Paths, Completion Checks, and Protocol Evolution

工程案例研究· Aervox-harness

摘要

持久记忆的删除标记不能保证派生内容已清除，清理也不能阻止工作者将先前读取的内容写回。本文研究本地单用户 SQLite 记忆仓储，将逻辑删除拆为来源失效、关联清理、写入守卫和完成验证。五个消融臂分别覆盖十个具名写入口的三个相对位置及五种清理遗漏，共 175 个固定案例。只清理产生 15 次错误完成和 10 次复活；只增加验证阻塞 15 次，仍有 10 次复活；只增加守卫仍漏报五种遗漏。守卫与验证结合时，30 次安全完成、5 次正确阻塞，所选矩阵内无错误完成或复活。写入口审查还发现来源仓储旁路，并以版本化证明重选处理验证增强前的完成记录。12 个生产路径 SIGKILL 探针及五种跨进程工作者调度在重试后均干净完成且保留对象不变；独立计时和三组合成规模描述本机成本。贡献是写入覆盖、失败文本及验证合同演化的工程案例，不构成任意并发正确性、模型遗忘、备份删除或物理擦除的证明。

关键词：持久记忆；逻辑删除；软件质量；SQLite；故障注入；可复现研究

Abstract

A deletion flag does not establish that derived memory content has been removed, and cleanup does not prevent a worker from persisting information fetched earlier. We study logical deletion in a local, single-user SQLite memory subsystem. Five component ablations cover ten named write entrypoints at three relative positions and five cleanup omissions, yielding 175 predetermined cases. Cleanup alone produces 15 incorrect completions and ten resurrections. Verification without guards blocks 15 cases but still permits ten resurrections; guards without verification miss five omissions. Their combination yields 30 safe completions and five correctly blocked omissions, with no incorrect completion or resurrection in the selected matrix. An API inventory exposes alternative provenance writes, and versioned evidence selection addresses completed requests that predate stronger checks. Twelve production-path SIGKILL probes and five cross-process worker schedules preserve retained data and reach clean completion after retry. Separate timing and three synthetic size profiles describe local operation costs. The contribution is a bounded engineering case concerning write coverage, failure text, and verification-contract evolution. It does not establish general concurrency correctness, model unlearning, backup erasure, or secure physical deletion.

1. 引言

面向长期交互的应用常将一条用户事实转换为多个持久化对象。原文进入主记录与历史修订，检索建立全文索引和向量，后台工作者生成摘要，事件系统保留调度载荷与失败原因。删除请求到来时，这些对象不一定与主记录处于同一生命周期。后台任务可能先获取正文，再在删除完成后保存正常结果或包含原文的异常消息。

删除质量因而包含两个时间条件：宣告完成时是否仍有受约束的逻辑残留，以及完成后的迟到写入能否使残留重新出现。清理语句的影响行数不能充分回答前者；在方法入口查一次来源存活状态也不足以回答后者，检查与写入之间可能发生删除，失败分支还可能绕过正常结果的守卫。

本文将这一问题视为随写入接口和验证规则共同演化的软件质量问题，提出四个研究问题。RQ1：清理、守卫和验证分别约束哪些失败模式，验证能否替代守卫？RQ2：按存储分类的测试会漏掉哪些含内容或可恢复引用的写入口？RQ3：验证条件增强时，如何使旧完成证明失效并重新检查？RQ4：所选生产竞争、进程终止和合成规模下，能够建立哪些恢复与成本证据？

本文报告三项实现经验。第一，错误文本、修订、来源证据与当前修订引用都是需要审查的写入口，仅列出待清理表不能说明覆盖完整。第二，验证检查当前状态，守卫约束后续持久化；验证通过后仍可发生回写。第三，旧合同下成功的证明不能自动代表增强后的验证合同，消费证明和选择待处理任务必须与验证器一起演化。我们用五臂消融、独立 SQL 观察、具名仓储入口和生产执行边界将这些经验连接到可复跑证据。

这些是既有删除和软件测试原则在单一系统中的工程应用。本文不提出新的删除定义、并发算法或通用证明框架，也不声称是这些原则的首次应用。可迁移的产物是审查问题、写入清单和实验结构；其外部有效性仍需在更多实现中检验。

2. 相关工作与定位

2.1 删除正确性与来源

数据库来源区分结果为什么存在及其值来自何处 [1]。本文利用显式来源关系定位删除对象，不提出新的来源理论，也不假设记录的边涵盖模型的全部语义影响。

最接近的系统先例是 DELF [7]：开发者在数据类型上标注删除规则，系统将其转换为异步操作，并以静态和动态检查发现遗漏。本文不重新声称发明删除验证，而是报告本地记忆仓储中的错误回写、来源入口旁路及验证版本演化。实验没有运行 DELF，不能形成效果或性能比较。

Athanassoulis 等区分应用与存储层的删除要求 [8]；P2E2 针对数据依赖约束删除后的推断 [9]。逻辑残留、物理保留和推断能力因此需要分别陈述。本研究只观察声明的逻辑存储和具名写入口，不测量后两类保证。

工作	主要对象	与本案例的关系
DELF [7]	注解、异步删除及规范质量检查	直接先例；本文提供具体写入口和证明演化的实现案例
删除合规数据系统 [8]	应用与存储层的持久删除	本文仅覆盖应用可查询状态
P2E2 [9]	依赖条件下的删除后推断	本文没有测量推断能力
本案例	写入口、失败文本、验证版本与恢复	单仓库实证，没有外部框架对照

Garg 等的形式化 [2] 讨论更广义的删除保证。本文的完成证明是应用检查结果，不是密码学证明，也不能独立证明未声明存储中的内容不存在。

2.2 受控测试与记忆遗忘

CHESS 通过受控调度重现并发错误 [3]，有界黑盒崩溃测试通过有限工作负载与故障注入检查一致性 [4]。本文借用显式调度与故障屏障，不枚举全部可达状态。定序矩阵、跨进程生产工作者及终止探针的实验单元不同，不合并计算可靠率。

Yao 等研究执行状态遗忘与恢复重放 [5]；MemLineage 通过来源和派生关联限制敏感动作 [6]。前者依赖确定性转移和可恢复环境等条件，后者受已记录父边与归因阈值约束。本文不测试模型、KV cache 或动作授权。[5]、[6] 按已核对的 arXiv v1 预印本引用，不表述为同行评审出版物。

3. 系统范围与完成条件

3.1 对象与逻辑存储

研究对象是 Aervox-harness 的本地单用户 SQLite Memory 仓储切片。主矩阵每库包含待删除对象 `target` 和应保持不变的对象 `keep`。清理保留身份墓碑，清空其正文和受约束引用。共享事实、其他生命周期拥有的派生投影由对应所有者处理；生产删除组件遇到不支持的目标时拒绝完成，实验不绕过该边界。

逻辑存储	清理后的条件
主记录	恰有一条删除墓碑；正文为空；关键词、来源、当前修订与父引用清空；验证状态失效
历史修订	目标全部修订的正文为空
来源证据	目标全部修订的证据为墓碑，来源范围清空
FTS	无目标全文索引项
向量	无目标嵌入项
压缩摘要	摘要为空且状态为墓碑
审计事件	目标事件的原因文本为空
Outbox	仅保留允许的身份字段及布尔删除标志；状态已发布；错误文本为空

身份字符串不计作已擦除内容；该假设要求其中不夹带正文。`snapshotId` 只允许字符串或空值。独立观察器比较 `keep` 在八类存储中的完整行快照，并在数据库外保存夹具中已知 Outbox 事件的归属，使丢失 `memoryId` 的畸形载荷仍可被观察。这个测量账本不是生产数据库已经具备的来源能力。

3.2 协议与前提

`checked(v)` 表示版本 v 验证器的一次成功读取；`completed(v)` 还要求结果已持久记录。

```
live -> invalidated + cleaned -> checked(v) -> completed(v)
                  |                  |
                  +-- crash/retry ---+-- residue -> blocked -> clean/recheck
completed(old-v) -> selected-for-recheck -> checked(current-v)
late writer ------> source check + write in one atomic boundary
```

`clean()` 在同一写事务中失效来源并清理该切片。`verify()` 重新读取数据库，不以影响行数代替检查。检查通过后，工作者先记录目标证明，再完成请求；检查拒绝时保持阻塞。事务提交、验证、目标证明与请求完成不是一个不可中断的操作，因此恢复必须依据持久状态重试。

这一协议依赖四项前提：所有适用持久化入口执行相应守卫；来源关联完整且身份字段不承载正文；其他入口不以同一身份重新激活墓碑；SQLite 事务为本地写入提供所需原子性。任意原始 SQL、插件、外部存储和外部副作用不在声明的写入边界内。共享投影属于另一生命周期，不能根据本切片通过推定其安全。实验检查枚举入口和屏障，不能从清单本身证明这些前提永久成立。

3.3 指标

令 C 为已宣告完成，R(t) 为八类逻辑存储的残留计数向量，K(t) 为保留对象快照未变。完成观察为 t_c，指定迟到写入后的观察为 t_f。

- 错误完成：C 为真，R(t_c) 至少一项非零。
- 完成后复活：C 为真，R(t_c) 全零，但 R(t_f) 至少一项非零。
- 安全完成：C 为真，R(t_c) 与 R(t_f) 均全零，且 K(t_f) 为真。
- 阻塞：验证拒绝完成。遗漏案例中的残留仍存在，不能计为成功删除。

这些是固定案例判定，不是概率估计。验证拒绝与写入拒绝也是不同事件：前者拒绝完成，后者防止迟到结果进入数据库。

4. 实现修正与覆盖清单

4.1 原子写入边界

清理更新墓碑、修订、证据、摘要和事件，清除 FTS 与向量，并把压缩 Outbox 改为不含正文的终态载荷。写入守卫必须与持久化共享原子边界。单条条件 SQL 可以把来源判断放在修改语句内；需要多次查询的来源仓储入口则在同一写事务中检查来源并插入。向量适配器通过生产嵌入仓储继承守卫。不同入口可返回拒绝、空结果或稳定错误码；实验以数据库观察判断安全性。

验证器再次检查八类存储。独立 SQL oracle 不调用生产 `verify()`，并通过残留正控制测试自身敏感性。两者共享删除范围和墓碑合同，仍可能共享未知遗漏，因此实现分离不等于完全独立的规格来源。

4.2 失败文本和来源仓储旁路

旧 Outbox 的 `markFailed` 与 `markDeadLetter` 仅按事件 ID 回写状态和错误文本。工作者先获取正文，删除再把事件变为终态墓碑后，迟到异常仍能恢复含正文的失败信息。修复把事件未删除及对应来源仍存活放入实际 UPDATE。下列示意省略重试计数，保留关键条件：

```
UPDATE outbox_events
SET status = 'dead_letter', last_error = :error
WHERE id = :event_id
  AND (
    event_type != 'memory.compaction.requested'
    OR (
      json_extract(payload, '$.deleted') IS NOT 1
      AND EXISTS (
        SELECT 1 FROM memory_records
        WHERE memory_records.id = json_extract(outbox_events.payload, '$.memoryId')
          AND memory_records.is_deleted = 0
      )
    )
  );
```

来源条件关联当前事件，不能由任意其他存活记录满足。若错误写入先发生，后续清理清除它；若清理先发生，更新条件不成立。这个两序解释仅适用于确实执行该原子条件的入口。

再次按公开入口审查发现，同一审计目的表还有 `recordMemoryEvent`，并且 `appendMemoryRevision`、`createMemoryEvidence` 能分别恢复修订正文和来源范围。只测已有守卫的 `recordEvent` 会漏过这些路径。本轮将来源检查与插入放入同一写事务；`setMemoryCurrentRevision` 的条件更新还要求来源存活且修订属于该来源，防止恢复已清空引用。删除标记更新保持单调，保留墓碑的主键约束拒绝同身份创建。

矩阵写入口	持久化目的地	守卫/覆盖依据
`indexMemoryFts`	FTS	条件写入；三个位置
`insertBatch`	向量	来源守卫；三个位置
`upsertMarker`	摘要	事务内来源检查；三个位置
`recordEvent`	审计原因	来源守卫；三个位置
`insertEvent`	Outbox	压缩事件来源守卫；三个位置
`appendMemoryRevision`	修订正文	同一写事务；三个位置
`createMemoryEvidence`	证据来源范围	沿修订查来源；同一写事务；三个位置
`recordMemoryEvent`	审计原因	同一写事务；三个位置
`setMemoryCurrentRevision`	主记录引用	条件更新；三个位置
向量适配器 `upsert`	向量	委托生产仓储；三个位置

此外，两个 Outbox 错误方法有独立旧/新比较；`createRecord` 和 `softDeleteRecord` 有墓碑及单调性回归；`clean` 由清理/遗漏矩阵覆盖。`createRecord(canonicalParentId)`、`createEdgeEvidence` 与 Skill 候选跨越共享投影所有权边界，以生产拒绝和边界回归处理，不算作主矩阵的完整删除。清单只覆盖这些具名仓储和向量适配入口，不宣称审计了任意 SQL 或插件。

4.3 验证增强与旧证明重选

旧验证器漏查部分引用、摘要状态和 Outbox 终态，并且只查一个正文键。增强的验证器核对完整墓碑，要求 JSON 布尔删除标志，拒绝额外键和嵌套 `snapshotId`；清理将畸形快照转为空值。

增强查询却沿用 `memory-local-v2` 会让旧完成记录绕过新检查。修订将合同集中定义为 `memory-local-v3`，让删除闸门、任务选择和证明消费共同识别旧版本、缺失或无效证明。工作者重新清理并生成当前证明，

而不是只改旧标签。回归从旧 v2 完成状态及残留载荷出发，检查闸门关闭、任务重选和 v3 完成。真实用户数据库是否已执行该重验仍需实际运行证据；本轮在合成数据库中验证升级路径。

5. 方法

5.1 五臂与固定调度

五臂共用夹具和预声明清单，无守卫路径和遗漏清理是研究 SQL 变异，不是五个外部产品或生产模式。

消融臂	来源及清理	写入守卫	完成验证
soft-delete	仅标记	无	无
cleanup-only	清理	无	无
verified-cleanup	清理	无	有
guarded-cleanup	清理	有	无
verified-guarded	清理	有	有

十个写入口各在清理前、清理后但完成前、完成后执行一次，共 30 个案例。五个附加案例各故意漏清 FTS、向量、摘要、审计或 Outbox 中的一个通道。每臂 35 个，总计 175 个；每个组合执行一次，每库相互独立。迟到内容取自删除前缓存载荷。`current-revision` 案例恢复引用而不是新正文，但违反同一墓碑合同，单独保留其入口身份。

这些位置通过顺序等待主动安排，不依赖偶然命中线程竞争；不能据此宣称真实并发已被穷尽。`verified-cleanup` 专门检验验证通过后仍可能回写，补足只比较三种防护组合时的解释缺口。

5.2 观察器正控制

oracle 在完成处及最终写入后读取八类存储，检查目标恰有一条墓碑，并比较 `keep`。证据查询沿全部目标修订关联。正常清理后逐类注入残留，确认检测能力；额外修订、来源字段丢失、畸形或扩展载荷、嵌套快照和保留对象证据破坏分别测试已知盲点。这些测试不加入主矩阵分母。

5.3 操作计时与规模

主计时每臂重复 24 次 `fts:before-clean`，共 120 条。每次使用新数据库文件和连接，进程与模块已经加载；按轮次轮换五臂顺序。计时范围为清理及可选验证，排除初始化、夹具、迟到写入和 oracle。每条记录同时保存完成及最终观察，避免把未完成操作混作成功耗时。只标记臂按设计具有不完整语义。

在该计时区间，`cleanup-only` 与 `guarded-cleanup` 执行相同清理函数；两者差异不能作为守卫成本。补充测量对存活来源的修订写入：旧 SELECT+INSERT 重建路径与生产事务路径各 12 次，交替先后，共 24 条。生产调用包含 JavaScript 抽象和事务成本，不将差值解释为守卫谓词的独立成本。

三种合成规模各启动三个新子进程，分别测清理和验证：

Profile	背景主记录	目标修订	配置内容字节
small	10	1	128
medium	1,000	100	1,024
large	10,000	1,000	4,096

每个额外目标修订附一条证据，并比较全部背景主记录的前后摘要。三个因素联合改变，未做单因素敏感性实验；向量/摘要扇出并未同比扩展。内容参数用于合成背景与新增修订/证据，不代表数据库中每个字段长度。新进程也不代表文件系统缓存冷启动。主表报告 type 7 插值分位数；仅三次的规模样本报告范围和中位数，不作显著性检验。

5.4 生产工作者与终止边界

原有四个错误回写案例比较 `markFailed`、`markDeadLetter` 的旧 SQL 与生产修复；两个既存终止探针分别检查手工事务和清理提交边界。它们保留在主报告中，但不代替新增生产路径探针。

补充报告在实际 `clean()` 的八次变更后、事务尚未提交时各终止一次，并覆盖清理提交后、目标证明写入前、目标证明写入后及请求完成后，共 12 个 SIGKILL 边界。观测代理在实际 awaited 数据库操作完成后通过 IPC 暂停子进程，不更改生产源码或事务边界。父进程在收到具名屏障后终止、重开，再运行真实删除工作者检查回滚、重选与幂等恢复。

五种跨进程调度调用真实压缩和删除循环：工作者取事件后删除、标记写入后删除、守卫持写锁时竞争清理、两个缓存同一事件的工作者，以及迟到异常。每个调度使用独立文件库和连接；双工作者调度启动两个子进程。持锁调度检查竞争清理确实遇到数据库忙后再放行并重试，其他调度以显式 IPC 屏障安排先后。这不是任意线程数的压力测试，也不模拟断电或任意 WAL 损坏。

5.5 证据与溯源

schema 3 主报告保存完整五臂清单、具名写入口、排除边界、当前验证版本和逐案例观察。补充报告单独保存终止、生产调度、规模与写入计时。报告记录生成时间、执行时 HEAD、工作区状态和关键源码/执行 JavaScript/schema/脚本/工具链哈希；主报告保存 SQLite 配置，规模报告另存引擎、客户端、内核和文件系统标识。未提交补丁也通过输入哈希标识，不能仅凭 HEAD 认定被测程序。

测量前后重新计算同一输入集，漂移或读取错误先保存原始证据再失败。校验器从原始观察重新计算分类与分位数，检查完整矩阵和 CSV 一致性；补充校验器检查边界、进程与恢复条件。当前源码一致性通过显式 `--check-source` 检查，与允许保留的历史归档报告区分。

6. 结果

6.1 完成时残留与完成后复活

消融臂	案例数	宣告完成	错误完成	完成后复活	安全完成	正确阻塞
soft-delete	35	35	35	0	0	0
cleanup-only	35	35	15	10	10	0
verified-cleanup	35	20	0	10	10	15
guarded-cleanup	35	35	5	0	30	0
verified-guarded	35	30	0	0	30	5



图 1：五臂在每臂 35 个固定案例上的结果。阻塞仍有残留，未算作成功删除。

只标记臂 35 次均错误完成。只清理臂能清除清理前的十次写入，但完成前的十次回写加五种遗漏形成 15 次错误完成；完成后的十次写入形成复活。新增 `verified-cleanup` 拒绝这 15 次有残留的完成，仍允许十次先通过验证、后复活。因此，完成验证无法替代后续写入守卫。

`guarded-cleanup` 没有观测到迟到回写残留，但五种遗漏仍被错误宣告完成。完整机制使这五种遗漏全部阻塞，其余 30 次安全完成。各臂最终有残留的案例依次为 35、25、25、5、5；验证的作用是拒绝不实完成，不会自行修复遗漏。175 个案例的保留对象均未改变，无未处理异常。结果回答 RQ1，同时说明表名相同的审计入口或不同向量入口需要分别覆盖，回答 RQ2 的覆盖问题。

6.2 实际修复与证明演化

四个 Outbox 探针中，两个旧 SQL 路径均在清理后恢复错误文本，分别返回 `failed`、`dead_letter`；两个生产修复路径均保持 `published` 墓碑且无正文残留。这里的 2/2 是两种具名方法的覆盖，不是故障概率。扩展主矩阵还覆盖修订正文、证据范围、来源审计和引用恢复，避免用已有守卫的单一路径代表整个目的表。

v3 升级回归从含旧证明和遗漏载荷的完成请求出发，检查闸门重新关闭、工作者重新选择、清理和新证明持久化。版本常量同时用于验证器、消费者和工作选择，回答 RQ3 的实现问题。这项证据来自合成状态回归，不是生产用户库的重验验收。

6.3 生产终止与跨进程调度

生产终止组	数量	重开观察与恢复
八次实际清理变更后、事务未提交	8	目标完整快照回滚；真实工作者各重试完成一次
清理提交后、目标证明前、目标证明后	3	已清理状态保留；真实工作者各重新完成一次
请求完成后	1	清理及完成状态保留；重试无需再完成请求

12 个探针均命中指定屏障并由 SIGKILL 终止。八个未提交变更后的目标快照与事务开始前一致；其余四个重开时已经干净。重试后全部干净，保留对象未变，完成闸门无待处理请求。主报告保留的两个代表性探针也通过，但不增加新的生产边界种类。

五种跨进程调度均最终完成一个删除请求，保留对象和目标状态通过 oracle。持锁调度观察到竞争清理的数据库忙，放行后生产写入先提交、后续清理完成；另四种调度的迟到工作者未留下含内容的活动摘要。双工作者调度中两个工作者均持有先前获取的事件，删除后仍无内容复活。这些结果回答 RQ4 在所列恢复与竞争边界上的部分，不能扩大为任意执行的证明。

6.4 操作耗时及合成规模

主报告时间字段为 2026-10-18 02:05:15（Asia/Shanghai），测量提交为 `bd19a16c`，含报告记录的未提交文稿改动。两份报告的 97 项关键输入前后哈希一致。环境为 Apple M5、macOS arm64、Node v24.21.0、libSQL 客户端 0.14.0、SQLite 3.45.1、Darwin 27.2.0；文件系统类型代码为 25。观察连接返回 `journal_mode=wal`、`synchronous=2`、`foreign_keys=1`、`busy_timeout=0`，这些是实测返回值，不以初始化配置推定连接状态。

主计时每臂 n=24，单位为毫秒，显示值保留三位小数，原始精度存于 JSON：

消融臂	P25	中位数	P75	P95
soft-delete	0.230	0.294	0.630	2.589
cleanup-only	1.329	1.755	2.445	4.594
verified-cleanup	3.842	4.623	5.921	8.465
guarded-cleanup	1.142	1.406	1.874	5.546
verified-guarded	4.116	5.210	6.057	7.946

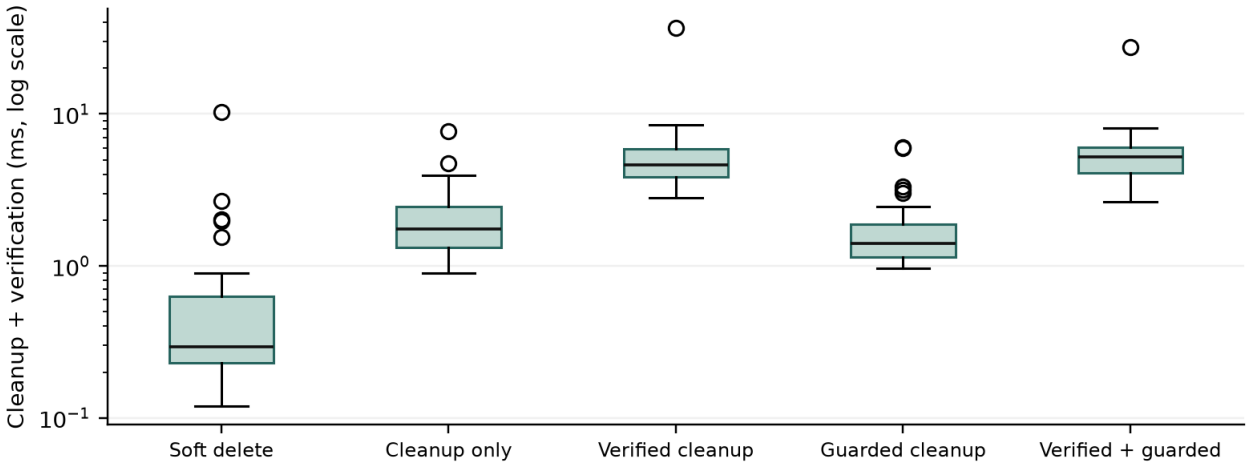


图 2：每臂 24 条清理及可选验证样本。采用对数纵轴描述分布。

完整机制在此小夹具的中位数为 5.210 ms；只清理为 1.755 ms。该差异包含验证读取，不包括迟到写入。只验证与完整机制的计时也不能用于估算守卫成本，因为守卫写入在计时区间之外。

三个新进程样本/profile 的结果如下，范围为该组三次观察的最小至最大值，均为毫秒：

Profile	清理中位数	清理范围	验证中位数	验证范围
small	2.194	1.895-3.770	3.221	2.725-5.064
medium	3.102	2.386-3.186	3.780	2.658-4.949
large	17.516	12.239-17.752	7.151	5.281-14.114

九个样本使用不同进程，目标最终干净；各 profile 的全部背景主记录摘要一致，保留对象完整。在 large profile 中，目标修订增至 1,000，清理中位数为 17.516 ms。背景规模、修订数量与内容长度同时变化，不能把成本增长归因于单一因素，也不能拟合可靠的扩展趋势。

存活来源的单列修订写入样本如下，单位为毫秒：

路径	n	P25	中位数	P75	P95
legacy-select-insert	12	0.296	0.335	0.398	0.458
guarded-production	12	0.422	0.624	0.743	2.053

24 次写入均实际持久化。生产 API 中位数为 0.624 ms，重建旧路径为 0.335 ms；两者包含不同的 JavaScript 和事务路径，样本仅描述本机调用，不是纯 SQL 谓词开销或端到端吞吐量。上述成本证据与正确性证据分开，不据此声称显著加速或几乎零成本。

7. 讨论与有效性

7.1 可迁移的质量审查

本案例支持三个相互独立的问题。“哪些存储需要清理”应与“哪些入口还能写回”并列审查；指向同一表的两个方法也可能采用不同守卫。其次，验证只观察当前状态，守卫约束后续受控写入；测量必须分别在完成时和迟到写入后取样。最后，验证合同变化属于持久协议演化，应同步处理证明、消费者和工作选择，不能只增强验证查询。

完成证明的版本不增加存储可见性。丢失的来源边、未声明的表或夹带正文的身份字符串仍可能造成漏检。未来需要独立规格、模式变异或跨实现复现扩大证据，不能把一次查询无结果当作范围完整性证明。

7.2 威胁与限制

内部有效性。 无守卫路径和遗漏是人为变异，不能代表生产故障频率。oracle 与验证器共享对象模型，可能共同遗漏。正控制、额外修订和完整保留快照只能减少已知盲点。

构念有效性。 安全完成和复活都是有限观察时刻的逻辑判定。阻塞仍可能含残留，不等于删除成功。引用恢复也按墓碑合同计为残留，不能把所有复活事件都描述为新正文写入。

外部有效性。 单仓库、合成数据和有限具名入口不代表跨项目、跨设备或复杂共享图。没有真实模型、外部记忆框架或用户库实验。共享投影所有者和任意原始 SQL 不在本切片保证中。

时序与故障有效性。 三个定序位置、五种生产调度及具名终止屏障不是穷尽状态空间。SIGKILL 保留操作系统和存储系统运行，不等于断电。IPC 代理也改变调度时机；它的用途是命中指定边界，而非重现自然故障频率。

成本有效性。 主计时排除守卫写入，补充写入计时包含整条 API 路径。规模 profile 同时改变三个因素，每组仅三次；背景记录和目标修订增长没有覆盖每种派生扇出。文件系统缓存、后台负载及单机环境限制外推。

7.3 研究材料与责任

实验使用新临时数据库，不读取真实用户内容。源码、工具链、JSON/CSV 和图表随研究包提供；耗时无需跨机器逐字节一致。作者、单位、贡献声明及归档 DOI 仅由实际取得的信息填写。当前版本是投稿准备材料，不表示已投稿或已被接受。复现、CI、PR 合入、真实数据库重验及发表各自需要证据。

8. 结论

本案例把本地记忆的逻辑删除分解为清理、受控写入和完成验证，并将验证合同演化纳入持久协议。写入口清单揭示正常结果以外的来源与失败文本路径；只验证臂仍有十次复活，完整机制形成 30 次安全完成与五次正确阻塞。生产终止和跨进程调度扩展了顺序小夹具的证据范围，但不能建立任意并发或跨系统保证。后续研究应在更多实现、来源关系完整性和共享投影所有权上验证这些审查方法，并将应用逻辑删除与物理删除、语义遗忘分开评估。

参考文献

[1] Peter Buneman, Sanjeev Khanna, Wang-Chiew Tan. Why and Where: A Characterization of Data Provenance. ICDT 2001, LNCS 1973, pp. 316-330. Springer, 2001. DOI: 10.1007/3-540-44503-X_20.

[2] Sanjam Garg, Shafi Goldwasser, Prashant Nalini Vasudevan. Formalizing Data Deletion in the Context of the Right to be Forgotten. Cryptology ePrint Archive, Paper 2020/254, 2020. 本文引用 ePrint 修订版。原文。

[3] Madanlal Musuvathi, Shaz Qadeer, Thomas Ball, Gerard Basler, Piramanayagam Arumuga Nainar, Iulian Neamtiu. Finding and Reproducing Heisenbugs in Concurrent Programs. 8th USENIX Symposium on Operating Systems Design and Implementation (OSDI 08), 2008. 会议记录。

[4] Jayashree Mohan, Ashlie Martinez, Soujanya Ponnapalli, Pandian Raju, Vijay Chidambaram. Finding Crash-Consistency Bugs with Bounded Black-Box Crash Testing. 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18), pp. 33-50, 2018. 会议记录。

[5] Chao Yao, Yangbo Wei, Zhen Huang, Junhong Qian, Chenle Chen, Shaoqiang Lu, Chen Wu, Lei He. Forgetting Without Restarting: Execution-State Unlearning for Stateful LLM Agents. arXiv:2609.04875v1, 2026. 预印本，版本 1。原文。

[6] Ciyan Ouyang, Rui Hou. MemLineage: Lineage-Guided Enforcement for LLM Agent Memory. arXiv:2605.14421v1, 2026. 预印本，版本 1。原文。

[7] Katriel Cohn-Gordon, Georgios Damaskinos, Divino Neto, Joshi Cordova, Benoît Reitz, Benjamin Strahs, Daniel Obenshain, Paul Pearce, Ioannis Papagiannis. DELF: Safeguarding

deletion correctness in Online Social Networks. 29th USENIX Security Symposium, 2020. 会议记录与原文。

[8] Manos Athanassoulis, Subhadeep Sarkar, Tarikul Islam Papon, Zichen Zhu, Dimitris Staratzis. Building Deletion-Compliant Data Systems. IEEE Data Engineering Bulletin, 45(1): 21-36, 2022. 作者研究组记录。

[9] Vishal Chakraborty, Youri Kaminsky, Sharad Mehrotra, Felix Naumann, Faisal Nawab, Primal Pappachan, Mohammad Sadoghi, Nalini Venkatasubramanian. Meaningful Data Erasure in the Presence of Dependencies. Proceedings of the VLDB Endowment, 18(10): 3435-3448, 2025. DOI: 10.14778/3748191.3748206。

附录 A：复现

复现入口见同目录 README；原始结果及补充报告是数值真源。以下命令从仓库根运行，依赖统一安装，工具链由 `mise.toml` 固定：

```
mise exec -- pnpm install --frozen-lockfile
mise exec -- pnpm exec turbo run build --filter=@aervox/worker... --concurrency=2
mise exec -- node --test scripts/memory-deletion-research/runner.test.mjs \
  scripts/memory-deletion-research/supplement.test.mjs
study_run="$(mktemp -d /tmp/aervox-memory-deletion.XXXXXX)"
mise exec -- node scripts/memory-deletion-research/runner.mjs \
  --out="$study_run" --repetitions=24
mise exec -- node scripts/memory-deletion-research/supplement.mjs \
  --out="$study_run/supplement.json" --repetitions=3
mise exec -- node scripts/memory-deletion-research/validate-report.mjs \
  "$study_run" --check-source
mise exec -- node scripts/memory-deletion-research/validate-supplement.mjs \
  "$study_run/supplement.json" --check-source
```

输出目录必须是新目录，禁止通过改写旧哈希或汇总数值令校验通过。图表从报告生成；正文数值、图表和原始数据须来自同一测量。Python 依赖固定于研究目录 `requirements.txt`，PDF 生成后仍须逐页检查。CI 通过不能替代真实数据库验收或正式发表。